

Here's the complete picture of the node-synchronization work, end to end.

1. The problem it exists to solve

Byte caching replaces a chunk of data with its 32-byte hash (a HIT) when the sender believes the receiver already holds that chunk. The original logic asked the wrong question:

"Do I have this chunk?" → if yes, send HIT.

That's only valid if both stores are identical. It breaks whenever they diverge — receiver was wiped, receiver is a new node, receiver has a smaller cache and evicted things, receiver

restarted with a stale file.

When it broke, the receiver's decode path hit goto reset: it printed a line and silently discarded the chunk's data. The file arrived with a hole in it and no error was raised anywhere. That is the failure the whole feature exists to eliminate.

—

2. Three mechanisms, layered

Each covers a different divergence pattern. They stack; none replaces another.

Mechanis m	Covers	Cost	Status

NAK	anything, including cases the others miss	1 RTT per failed chunk	built, working
Epoch	peer wiped / new incarnati on	one 7- byte message per connectio n	built
Confirme d-set	peer has <i>a partial</i> subset	~160 KB per peer	not built

3. Wire protocol

Every message is a TLV: type(1) | len_hi(1) | len_lo(1) | value(len).

Type	Name	Value	Direction
0xfd	HIT	hash (32 B)	sender → receiver
0xbf	MISS	data (N B)	sender → receiver
0xfb	NAK	hash (32 B)	receiver → sender

0xfe	HELLO	generation (4 B, big-endian)	both, once per connection
------	-------	------------------------------	---------------------------

0xfb and 0xfe are the two additions.

4. Mechanism 1 – NAK

Receiver side. A HIT arrives, hash isn't in the local store. Instead of dropping the data, build a NAK carrying that hash and send it back on so1.

The so1 plumbing was the real work here.
module4_decode_tlv_chain originally only

received so2 — the *forwarding* socket, pointing onward to the LAN client. A reply has to go back the way it came, which is a different socket. so1 was already in scope at the call site in uipc_socket.c (assigned at line 745), just never passed down. Adding it changed the function signature from 3 parameters to 4.

Sender side. NAK arrives. Look up that hash in *our* store. Found → rebuild the chunk and send it as a MISS on so1. Not found → both sides have lost it; increment nak_unsatisfiable and log loudly. Nothing is synthesized, nothing is silently dropped.

Property: never corrupts, at any scale, under any divergence. That's why it stays permanently as the floor under everything else.

Limit: it discovers staleness one bounced chunk

at a time. A GB-scale store wiped clean means ~[1.3](#) million individual failed HITs, each costing a round trip on a 700 ms link.

5. Mechanism 2 – Epoch / generation

The idea: instead of learning the peer is stale a million times, learn it once, before sending anything.

The generation number. Each store carries an incarnation ID, persisted in a sidecar file `Chunk_store.gen` next to `Chunk_store.dat`.

In `module2_init`, the existence of the store file *is* the wipe signal. If `fopen(..., "r+b")` fails and the file has to be created, the store's contents are

gone — so a new generation is assigned and written to the sidecar. If the store opened normally, the existing generation is read back unchanged, so an ordinary process restart does not look like a wipe.

Why time-based, not a counter. A counter (previous + 1) fails in two real cases: the whole directory gets wiped so the counter restarts at 1, or new hardware reuses a decommissioned node's tunnel IP and also starts at 1. Either way the peer compares 1 against its remembered 1, sees no change, and resumes sending HITs that all bounce. `time(NULL)` gives a value distinct from the previous incarnation with no memory required. Collisions *between* nodes are harmless — the peer table is keyed per peer, so only "same peer, different incarnation" has to differ.

HELLO. In `module1's process_chunk`, before the

first chunk's TLV is queued on a connection, a HELLO carrying our own generation is appended first. A `hello_sent` flag in `module1_state` makes it exactly once per connection; `module1_state_init` memsets the struct, so a new connection re-sends it.

Ordering matters and the simulator proved it: the same restart scenario gave 0 NAKs with the epoch resolved at session start, and 158 NAKs with it piggybacked on ongoing traffic. Learning late is nearly worthless.

The `peer table.module4.c` holds `g_peers[MODULE4_MAX_PEERS]` — each entry `{peer_key, generation, is_cold, in_use}`, keyed by the peer's tunnel IP (`t_tcblut->gre_dstIP`, already populated at connection setup in `tcp_input.c`).

When a HELLO arrives:

generation differs from what's recorded →
the peer's store is a different incarnation →
mark `is_cold = 1`

generation matches → our knowledge is still
valid → `is_cold = 0`, HITs re-enabled

peer not in table at all → unknown → cold.

This is the safe default and it's what makes
a brand-new node correct with no special
handling.

`Enforcement.module4_lookup_and_build_tlv`
gained a `peer_cold` parameter. Its decision
became:

c



```
if (cid != 0 && !peer_cold) → HIT  
else → MISS
```

A local store hit is only usable if the peer can also resolve it. When cold, the chunk goes as MISS directly — the wasted round trip never happens. `forced_miss_cold` counts these so you can see the mechanism working.

The RTT backstop. HELLO can't reach the sender until one RTT after the transfer starts, so on a 700 ms link the sender would fire HITs into that gap. A received NAK therefore *also* marks the peer cold. That caps the bounces at roughly one RTT's worth instead of letting them continue for the whole file.

6. File by file

module2.h / module2.c — generation field, Chunk_store.gen persistence, module2_get_generation(). Also a bounds check added to find_by_hash so a corrupted slot index returns a miss rather than reading out of range.

module1.h / module1.c — module1_state gains peer_is_cold and hello_sent. append_pending() factored out (HELLO and chunk TLVs both need it). process_chunk prepends HELLO once, then passes s->peer_is_cold down to module4.

module4.h / module4.c — HELLO and NAK types, the peer table and its accessors, module4_build_hello(), peer_key_of(), the peer_cold parameter, HELLO/NAK cases in handle_record, and the stats counters.

uipc_socket.c — two changes total: so1 added to

the `module4_decode_tlv_chain` call, and one line stamping `tp1->m1_state.peer_is_cold` before `module1_add_data`.

7. Three design decisions worth knowing

HELLO is a TLV, not a TCP option. The plan called for piggybacking on `session_dedup_flags` in `tcp_finalize_options()`. That field appears exactly once in the entire codebase — a declaration in `tcp_var.h` with an aspirational comment. Nothing sets it, nothing reads it. There was no negotiation to piggyback on. A TLV reuses the machinery NAK already proved works and avoids surgery on SYN option

encoding.

Peer state lives in module4, not TCPlookuptable. Keying the table on the already-populated gre_dstIP avoided modifying a 55 KB core header.

peer_is_cold lives in module1_state. The plan assumed process_chunk could reach tp->tcblut. It can't — it only receives struct module1_state *. The caller in uipc_socket.c does have tp1 in scope, so it stamps the flag on the way in.

8. What is not built

The confirmed-set bitmap. This is the gap you

identified yourself: peer has chunks 800–1000 but not 1–799. Epoch is a single number — it can only say "all valid" or "all invalid," and both answers are wrong for a partial subset. Handling that needs one bit per chunk (~160 KB per peer for [1.3](#) M chunks), plus a CONFIRM message for the peer to report what it stored, plus sender_cid on MISS so both sides share an ID space.

Also unbuilt: eviction notify, Bloom-filter bulk reconcile, and persistence of the peer table across a hub restart (currently in-memory only — a hub restart makes every peer cold, which is wasteful but safe).

Today, partial divergence is handled correctly but inefficiently — NAK catches every case, one round trip at a time.